BARQ ACADEMY

DevOps Internship – Project Submission

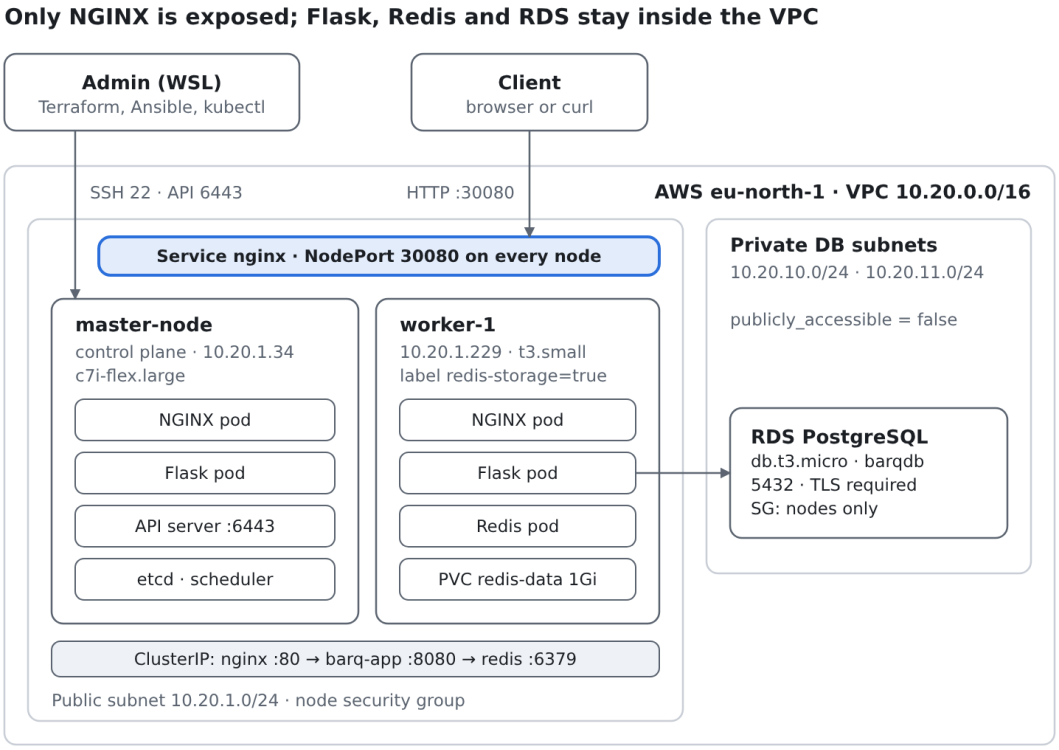
SUBMISSION DOCUMENT | BARQ ACADEMY

INTERN NAMES	Zinab Mohamed Elsayed Mazen Hassan
TASK / ASSIGNMENT	Automated AWS Infrastructure and Kubernetes Deployment

BARQ on AWS: Terraform, Ansible and Kubernetes

Terraform builds the AWS network, two EC2 nodes and a private RDS PostgreSQL; Ansible turns the nodes into a two-node kubeadm cluster; the BARQ app runs on it behind NGINX with Redis on a persistent volume. Every check in section 4 of the assignment was run on the live cluster and passed; the one honest limit is that Redis storage does not survive loss of the worker node (see Redis storage limitation).

Architecture



Architecture: VPC, two nodes, RDS, request path

A client reaches NGINX only on NodePort 30080; NGINX and Flask run as two pods each, spread over both nodes, Flask uses one Redis pod (its volume sits on worker-1) and RDS in private subnets on port 5432 over TLS. The admin workstation reaches the nodes only on SSH 22 and the API on 6443 from the approved IP range, and both nodes pull the application image from a public registry.

Environment and decisions

The stack runs in eu-north-1 on two EC2 nodes plus one managed database; each choice below was made for a reason the assignment or the account imposed.

Area	Choice	Why
Region	eu-north-1 (Stockholm)	The only region the account's policies allow

Area	Choice	Why
Control plane	c7i-flex.large, Ubuntu 24.04, 20 GB gp3 encrypted	Eligible instance type with enough memory for kubeadm and system pods
Worker	t3.small, Ubuntu 24.04, 20 GB gp3 encrypted	Eligible instance type; runs app, NGINX and Redis
Kubernetes	v1.31.14 (kubeadm), containerd 2.2.1	Same version on both nodes, packages held so they do not drift
Pod network	Calico v3.27.0 in VXLAN mode, pod CIDR 10.244.0.0/16	Enforces NetworkPolicy; VXLAN works on any cloud network
Database	RDS PostgreSQL, db.t3.micro, private subnets, storage encrypted, TLS required	Not reachable from the internet; only the node security group may connect on 5432
Terraform state	S3 bucket with versioning and public access blocked, state locking on	Shared, recoverable state instead of a local file
Persistent storage	local-path-provisioner, StorageClass `local-path`	Simplest option on kubeadm without a cloud CSI driver (limits in Redis storage limitation)
Image	`docker.io/zinabelamir/barq-app` (public), fixed tags `6ca3443`, `v2`, `v3`	Fixed tags make rollout and rollback exact; `latest` is never used
Public entry	NGINX Service of type NodePort 30080	Only NGINX is exposed; no load balancer cost
Admin access	SSH 22 and API 6443 only from the approved client range; SSM Session Manager through an instance role	No world-open management ports

Terraform outputs from the applied stack:

Output	Value
control_plane_public_ip / private	51.20.123.248 / 10.20.1.34
worker_public_ip / private	56.228.9.58 / 10.20.1.229
rds_endpoint	barq-postgres-lab.cjaqwcigslcf.eu-north-1.rds.amazonaws.com:5432
rds_database_name	barqdb

Repository layout

Everything for this task lives on branch `aws-terraform-ansible`; nothing secret is committed.

```
terraform/
  versions.tf      provider and S3 backend
  variables.tf     inputs (validated)
  main.tf          VPC, subnets, routes, security groups, IAM, 2 EC2, RDS
  outputs.tf       IPs, RDS endpoint, SSH commands
  terraform.tfvars.example   safe example (copy to terraform.tfvars, never commit)
Ansible/
  generate-inventory.sh      builds hosts.ini from terraform outputs
  k8s-setup.yml              kubeadm cluster: common, control plane, worker, verify
manifests/
  deploy.sh                  creates the Secret from .env, applies everything, waits
  .env_example               DB_USER / DB_PASSWORD placeholders
  k8s/00-namespace.yaml ... 05-nginx/   namespace, config, db-init Job, Redis, Flask, NGINX
  k8s/06-networkpolicy.yaml  ingress allow-lists
  k8s/kustomization.yaml     one apply for all of the above
app/ , Dockerfile , database/init.sql    the BARQ service and its schema
```

Deployment steps

Run the steps in order from a WSL or Linux shell that has the AWS CLI, Terraform, Ansible and kubectl; each step ends with the check that proves it worked.

1. Prepare AWS access and the state bucket (once)

```
aws sts get-caller-identity          # confirms the CLI talks to the right account
aws s3api create-bucket --bucket barq-terraform-state-mazen --region eu-north-1 \
  --create-bucket-configuration LocationConstraint=eu-north-1
aws s3api put-bucket-versioning --bucket barq-terraform-state-mazen --versioning-configuration
Status=Enabled
aws s3api put-public-access-block --bucket barq-terraform-state-mazen \
  --public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

An EC2 key pair named `project` must already exist, with its private key at `~/.ssh/project.pem` (mode 600).

2. Terraform: network, nodes and database

```
cd terraform
cp terraform.tfvars.example terraform.tfvars   # edit: ssh_allowed_cidr, key_pair_name, db_password
terraform init           # use -migrate-state if a local state already exists
terraform plan -out tfplan
terraform apply tfplan
terraform output
```

`ssh_allowed_cidr` is your public IP with `/32`. `db_password` needs 12+ characters, letters and digits only, because the password is placed in a connection URL. The apply creates 19 resources (VPC, public subnet, two private DB subnets, internet gateway, route tables, two security groups, IAM role and profile, two EC2 nodes, RDS subnet group and instance). Check: `terraform output` prints the IPs and the RDS endpoint listed in Environment and decisions.

3. Generate the Ansible inventory

```
cd ../Ansible
./generate-inventory.sh ~/.ssh/project.pem
```

The script reads `terraform output` and writes `hosts.ini` (groups `master` and `workers`, user `ubuntu`, key path). No IP is typed by hand, and `hosts.ini` is git-ignored because it holds the current IPs.

4. Ansible: build the cluster

```
ansible-playbook -i hosts.ini k8s-setup.yml
```

The playbook waits for SSH, then on both nodes disables swap, loads `overlay` and `br_netfilter`, sets the sysctls, installs containerd with the systemd cgroup driver and pins kubelet, kubeadm and kubectl 1.31. On the control plane it runs `kubeadm init` (API certificate includes the public IP), installs Calico in VXLAN mode only if it is not already installed, and creates a join command; the worker joins with it; the last play waits for both nodes to be Ready. First run: master `changed=25`, worker `changed=15`, `failed=0`. Immediately re-running it gives `changed=0` on both nodes, because every step is guarded (`creates:`, conditions, and a check for an existing Calico daemonset).

5. kubectl from WSL

Copy the admin kubeconfig from the control plane (for example `ansible master -i hosts.ini -m fetch -a "src=/etc/kubernetes/admin.conf dest=~/.kube/barq-aws.conf flat=yes" -b`), point `server:` at `https://<control-plane-public-ip>:6443`, then:

```
export KUBECONFIG="$HOME/.kube/barq-aws.conf"
kubectl get nodes -o wide      # master-node and worker-1 Ready, v1.31.14
```

Port 6443 is open only to the approved client range. If your public IP changes, update `ssh_allowed_cidr` and run `terraform apply`; only the security group changes, the cluster is untouched.

6. Storage class and node label

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/<version>/deploy/local-path-storage.yaml
kubectl get storageclass                    # local-path, WaitForFirstConsumer
kubectl label node worker-1 redis-storage=true    # Redis is pinned to this node
```

7. Deploy the application

```
cd ../manifests
cp .env_example .env     # set DB_USER and DB_PASSWORD (the RDS master credentials)
./deploy.sh
```

`deploy.sh` creates the namespace, builds the `barq-secret` Secret from `.env` (the file is git-ignored), applies the kustomization, waits for the `db-init` Job to complete and for Redis, Flask and NGINX to roll out, then prints pods, services and the PVC. Check: `curl http://<node-public-ip>:30080/ready` returns `{"status":"ready","dependencies":{"postgres":"ready","redis":"ready"}}`.

8. Build and publish a new application image

```
podman login docker.io          # once, with your registry account
podman build --platform linux/amd64 -t <registry>/barq-app:<new-tag> .
podman push <registry>/barq-app:<new-tag>
kubectl -n barq-project set image deployment/barq-app app=<registry>/barq-app:<new-tag>
kubectl -n barq-project rollout status deployment/barq-app
```

Use a tag that no node has pulled before: the Deployment uses `imagePullPolicy: IfNotPresent`, so a reused tag would silently run the cached image. Podman is used instead of Docker: it builds the same Dockerfile, needs no background daemon, and the image name should be fully qualified (for example `docker.io/zinabelamir/barq-app:<new-tag>`) so Podman does not have to guess the registry.

Kubernetes design

A request enters on NodePort 30080, passes through one of two NGINX pods to one of two Flask pods, which read and write RDS PostgreSQL and a single Redis pod; all objects live in the `barq-project` namespace and apply with one `kubectl apply -k manifests/k8s`.

Manifest	Objects	Notes
00-namespace.yaml	Namespace barq-project	

Manifest	Objects	Notes
01-config/configmap.yaml	ConfigMap barq-config	DB_HOST (RDS endpoint), DB_PORT 5432, DB_NAME barqdb, DB_SSLMODE require, REDIS_URL redis://redis:6379/0, APP_PORT 8080, APP_MESSAGE
Secret (not in Git)	barq-secret with DB_USER, DB_PASSWORD	Created by deploy.sh from the git-ignored .env
02-db-init/	ConfigMap db-init-sql, Job db-init	postgres:16-alpine runs the schema over TLS; seed rows are inserted only if the table is empty, so reruns are safe
03-redis/	PVC redis-data (1 Gi, RWO, local-path), Deployment redis, Service redis	redis:7.4-alpine, AOF on, strategy Recreate, nodeSelector redis-storage=true
04-flask/	Deployment barq-app (2 replicas), Service barq-app (ClusterIP 8080)	Rolling update with maxUnavailable 0, maxSurge 1; tolerates the control-plane taint; preferred anti-affinity spreads replicas across nodes
05-nginx/	ConfigMap nginx-conf, Deployment nginx (2 replicas), Service nginx (NodePort 30080)	nginx:1.28-alpine, upstream barq-app:8080, retries the other replica on error or timeout
06-networkpolicy.yaml	nginx-ingress, app-ingress, redis-ingress	Anyone may reach NGINX on 80; only NGINX and Flask pods reach Flask on 8080; only Flask reaches Redis on 6379
kustomization.yaml	Lists all of the above	Pins the app image tag with an images: entry

Health and sizing:

Workload	Liveness	Readiness	Requests / limits
Flask	/health, 10 s delay, every 10 s	/ready (checks PostgreSQL and Redis), 5 s delay, every 5 s	100m, 128 Mi / 500m, 256 Mi
NGINX	/nginx-health, 10 s delay, every 10 s	/nginx-health, every 5 s	50m, 32 Mi / 200m, 64 Mi
Redis	redis-cli ping, 15 s delay, every 10 s	redis-cli ping, every 5 s	50m, 64 Mi / 250m, 256 Mi

Because readiness depends on /ready, a Flask pod that loses RDS or Redis is taken out of the NGINX pool instead of returning errors. DATABASE_URL is assembled inside the pod from the ConfigMap and Secret values, so the password never appears in a manifest. Running deploy.sh or kubectl apply -k resets the image to the tag pinned in kustomization.yaml; during a rollout test use kubectl set image instead.

Verification results

All nine checks in section 4 of the assignment passed on the live cluster on 15–16 October 2026; "Evidence" gives the figure number in the Screenshots section. Requests went to http://51.20.123.248:30080 (NodePort), so every test also exercises NGINX.

Required check	What was run	Result	Evidence
Test /, /health, /ready, /instance, /records, /counter	curl each path through the NodePort	All HTTP 200; /ready reports postgres: ready and redis: ready; version 2.1.0	Fig. 15, 16

Required check	What was run	Result	Evidence
/records creates and reads RDS data	POST /records with title proof-rds-1, then GET /records, then psql from a temporary postgres:16-alpine pod straight to RDS with sslmode=require	POST returned id 3; SELECT * FROM records on RDS lists the same 3 rows	Fig. 17, 18
Record survives Flask pod recreation	Deleted both barq-app pods, waited for rollout, GET /records	New pods served all 3 rows including proof-rds-1	Fig. 19, 20
Counter survives Redis pod recreation	/counter returned 4, 5; deleted the Redis pod (PVC left alone); /counter returned 6; repeated 7, 8, then 9 after a second deletion	Counter continued instead of resetting; PVC redis-data stayed Bound to the same volume	Fig. 21–24
Both Flask replicas answer through NGINX	12 and then 200 repeated GET /instance	Responses alternate between the two pod names	Fig. 25, 27
Replacement after deleting a Flask pod	Deleted one barq-app pod, listed pods	Deployment created a new pod; 2/2 Running again	Fig. 26 (also Fig. 19)
Service DNS and cross-node communication	Temporary busybox pods pinned to master-node and to worker-1 ran nslookup and wget http://barq-app:8080/instance; Flask pods called each other by pod IP across nodes	CoreDNS (10.96.0.10) resolves both Services from both nodes; pod-to-pod calls across nodes succeed (output below)	Fig. 28–30 and output below
Update the image, verify, roll back, verify	Baseline 6ca3443 (2.1.0); built v3 (version bumped to 2.2.0), kubectl set image, rollout status; then kubectl rollout undo	All requests returned 2.2.0 after the update and 2.1.0 after the rollback; rollout history records both steps; no downtime	Fig. 31–36
Ansible rerun keeps the cluster	Second ansible-playbook run on the working cluster	changed=0, failed=0 on both nodes (first run was 25 and 15 changes); Calico install skipped	Fig. 10, 11

Supporting checks:

Check	Result	Evidence
Nodes ready	master-node and worker-1 Ready, v1.31.14; all kube-system pods Running	Fig. 12, 13
Infrastructure exists as coded	VPC, three subnets, route tables, internet gateway, two EC2 instances (3/3 status checks), Terraform outputs	Fig. 1–5, 7
Network policies applied	app-ingress, nginx-ingress, redis-ingress created; afterwards /ready still returns ready inside a pod and the NodePort still answers 200	output below

Cross-node pod communication and network-policy check (terminal output, pod on master-node calling a pod on worker-1):

```
$ kubectl get pods -n barq-project -l app=barq-app -o wide
barq-app-86dccb6656-cbg85   1/1   Running   10.244.226.83   worker-1
barq-app-86dccb6656-qjcr9   1/1   Running   10.244.77.143   master-node

$ kubectl exec -n barq-project barq-app-86dccb6656-qjcr9 -- python -c "import socket;
print(socket.gethostbyname('barq-app.barq-project.svc.cluster.local'))"
10.102.233.31

$ kubectl exec -n barq-project barq-app-86dccb6656-qjcr9 -- python -c "import urllib.request;
print(urllib.request.urlopen('http://10.244.226.83:8080/health', timeout=5).read().decode())"
{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"alive","version":"2.1.0"}
```

```
$ kubectl exec -n barq-project nginx-6b69b87684-ldsgw -- wget -qO- http://barq-app:8080/health
{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"alive","version":"2.1.0"}
```

Redis storage limitation

Redis data survives pod deletion and restart, but it does not survive losing or replacing `worker-1`: pod recovery was proven, node-failure resilience was not and is not claimed.

How it is built. The PVC `redis-data` (1 Gi, ReadWriteOnce, StorageClass `local-path`) is served by local-path-provisioner, which creates a directory on the disk of the node where the pod first runs (binding mode `WaitForFirstConsumer`). The resulting PersistentVolume is tied to `worker-1`, and Redis is pinned there with `nodeSelector redis-storage=true` so the pod always lands next to its data. Redis writes an append-only file (`appendonly yes`) plus a snapshot every 60 s when at least one key changed. The Deployment uses `Recreate` because the volume is ReadWriteOnce. The StorageClass default reclaim policy is `Delete`, so the PV was patched to `Retain`: deleting the PVC by mistake no longer deletes the data.

Event	Data survives?	Why
Redis pod deleted or crashed	Yes	New pod mounts the same PVC; shown by the counter test
Rollout, `kubectl rollout restart`	Yes	Same volume
`worker-1` reboots	Yes	The directory is on its persistent EBS root volume
PVC deleted	Yes, PV kept (`Retain`)	Data stays on disk until the PV is removed by hand
`worker-1` terminated, replaced or its disk lost	**No**	Data lives only on that node; the PV cannot attach elsewhere and the pod stays Pending
`terraform destroy`	No	Nodes and disks are deleted

Impact is small by design: `/counter` is a cache-style value, while the records live in RDS, which is separate from both nodes. To remove the limit, use one of: an EBS CSI driver volume (survives node replacement inside one availability zone), a managed ElastiCache for Redis, or a scheduled copy of the AOF file to S3.

Security

No credential is stored in Git, the database is unreachable from the internet, and only NGINX is exposed; three gaps remain and two earlier ones are closed (below), so none is mistaken for a feature.

Area	What is in place
Secrets	Database credentials exist only as the Kubernetes Secret `barq-secret`, created at deploy time from a git-ignored `.env`; the RDS password is supplied through a git-ignored `terraform.tfvars` and only the example file is committed
Terraform state	Kept in a private, versioned S3 bucket with public access blocked; it contains the database password, so the bucket must stay private
Exposure	Only the NGINX NodePort 30080 is reachable from outside; Flask and Redis are ClusterIP Services
Pod traffic	NetworkPolicies allow NGINX → Flask 8080 and Flask → Redis 6379 only
Database	Private subnets, `publicly_accessible = false`, security group allows 5432 only from the node security group, storage encrypted, TLS required (`sslmode=require`)
Nodes	SSH 22 and API 6443 only from the approved client range; IMDSv2 required; root volumes encrypted; SSM

Area	What is in place
	Session Manager available through an instance role
Container	Image runs as non-root user 10001 on a digest-pinned base image; CPU and memory limits on every workload

Known gaps:

1. NodePort traffic is plain HTTP; there is no TLS termination.
2. Kubernetes Secrets are base64 in etcd and not encrypted at rest.
3. local-path-provisioner v0.0.30 is installed; later releases (v0.0.34 and newer) carry a security fix and should be used.

Closed gaps:

- The app no longer logs DATABASE_URL; it logs only whether the database and Redis are configured, so the password no longer reaches the container log.
- The Flask Deployment has a securityContext: non-root user 10001, read-only root file system, no privilege escalation, all capabilities dropped (Figures 37 to 39).

Troubleshooting

These are the problems actually met during this deployment, with the cause and the fix.

Symptom	Cause	Fix
Ansible rerun shows `changed=8` on the control plane	`get_url` with `force: false` still re-downloads the Calico manifest, so the patch and apply tasks ran again	Check for an existing `calico-node` daemonset first and install Calico only when it is missing; rerun then reports `changed=0`
`'calico_check' is undefined`	The guard task was removed or moved below the task that uses it	Keep "Check whether Calico is already installed" before the guarded block
kubectl from WSL times out or is refused	Your public IP changed and the security group no longer allows 6443	Set the new `ssh_allowed_cidr` and run `terraform apply`; only the security group changes
kubectl reports a certificate name mismatch	The API certificate does not list the address used	The playbook adds the public IP as an extra SAN at `kubeadm init`; fall back to `--tls-server-name=kubernetes`
`curl: (3) URL using bad/illegal format`	`$APP_URL` was empty	`export APP_URL=http://<node-public-ip>:30080`
`pods "10.244.x.x" not found`, or `URL can't contain control characters`	`kubectl exec` takes a pod name, not an IP; a stray space was inside the URL	Pass the pod name, then `http://10.244.x.x:8080/health` with no space
`NXDOMAIN` lines in `nslookup`	The resolver tries every search domain; only the `.svc.cluster.local` answer counts	Normal, no action
New image tag deployed but old version answers	`imagePullPolicy: IfNotPresent` kept the cached image for a reused tag	Use a new tag for every build
Image reverts after `deploy.sh` or `kubectl apply -k`	`kustomization.yaml` pins the tag	Use `kubectl set image` for rollout tests, or change the pinned tag
Redis pod or PVC stays Pending	Node is not labelled, or the StorageClass is missing; the PVC also waits until the pod is scheduled (`WaitForFirstConsumer`)	Install local-path-provisioner and run `kubectl label node worker-1 redis-storage=true`

Symptom	Cause	Fix
RDS connection errors with a password containing /, @ or spaces	Characters break the DATABASE_URL and RDS forbids some of them	Alphanumeric passwords only; Terraform validates this
git push rejected (fetch first)	Remote branch has commits you do not have	git pull --rebase origin <branch>, then push
git pull --rebase refuses to run	Staged or unstaged changes	Commit or stash first
Large kubectl binary and hosts.ini show up in Git	Generated files were committed	git rm --cached them and add them to .gitignore

Cleanup

Remove things from the inside out so nothing keeps billing: application, volume, infrastructure, then the state bucket.

1. Delete the application and its volume:

```
kubectl delete namespace barq-project
kubectl get pv                  # the Redis PV is kept (Retain); remove it
kubectl delete pv <pv-name>
kubectl delete -f https://raw.githubusercontent.com/rancher/local-path-provisioner/<version>/deploy/local-path-storage.yaml
```

2. Destroy the infrastructure (about 5 to 10 minutes, RDS is the slow part; no final snapshot is taken, so the database is gone for good):

```
cd terraform && terraform destroy
```

3. Empty and delete the state bucket only after step 2 has finished: remove every object version, then aws s3api delete-bucket --bucket barq-terraform-state-mazen --region eu-north-1.

4. Delete local files that hold secrets or IPs: Ansible/hosts.ini, manifests/.env, terraform/terraform.tfvars, ~/.kube/barq-aws.conf; remove the EC2 key pair project and any pushed image tags you no longer need.

5. In the AWS console confirm that no EC2 instance, EBS volume, RDS instance or Elastic IP is left in eu-north-1.

Evidence index

All 36 screenshots are embedded in the Screenshots section below and numbered Figure 1 to Figure 36 in the order they appear.

Figure	Shows
1–4	AWS console: VPC, route tables, subnets, internet gateway
5	Two EC2 instances running, 3/3 status checks
6	terraform init migrating state to the S3 backend
7	Terraform apply complete and outputs

Figure	Shows
8	Node security group rules
9	`generate-inventory.sh` writing `hosts.ini`
10, 11	Ansible first run (changed 25 / 15) and rerun (changed 0 / 0)
12, 13	`kubectl get nodes -o wide` and `kube-system` pods
14	local-path-provisioner install, worker label, `deploy.sh`
15	First application checks through the NodePort
16	All six endpoints answering 200 through the NodePort
17, 18	Record created through the API, then read straight from RDS with `psql`
19, 20	Both Flask pods deleted and recreated; records still returned
21–24	Redis pod deleted twice; counter continues 6 and 9; PVC `redis-data` still Bound
25, 27	Requests alternating between the two Flask replicas through NGINX
26	One Flask pod deleted and replaced
28–30	Service DNS from busybox pods pinned to each node
31	Application version changed from 2.1.0 for the update test
32–34	Image `v3` built, pushed, rolled out; all responses report 2.2.0
35, 36	`rollout undo`, image back to `6ca3443`, responses report 2.1.0, rollout history

Screenshots

The same 36 screenshots as the index above, grouped by what they prove; “Figure N” is the number used in the Evidence column of the verification tables.

A. Infrastructure built by Terraform

Figure 1. VPC `barq-aws-assignment-vpc` in state Available.

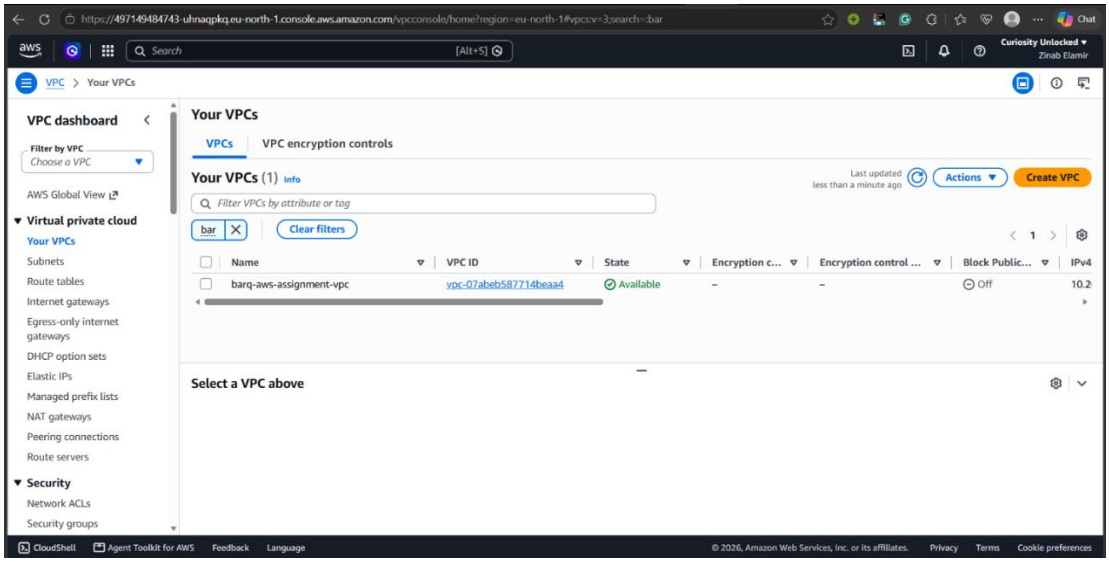


Figure 1: VPC

Figure 2. Route tables: a public one, and a private one for the two database subnets.

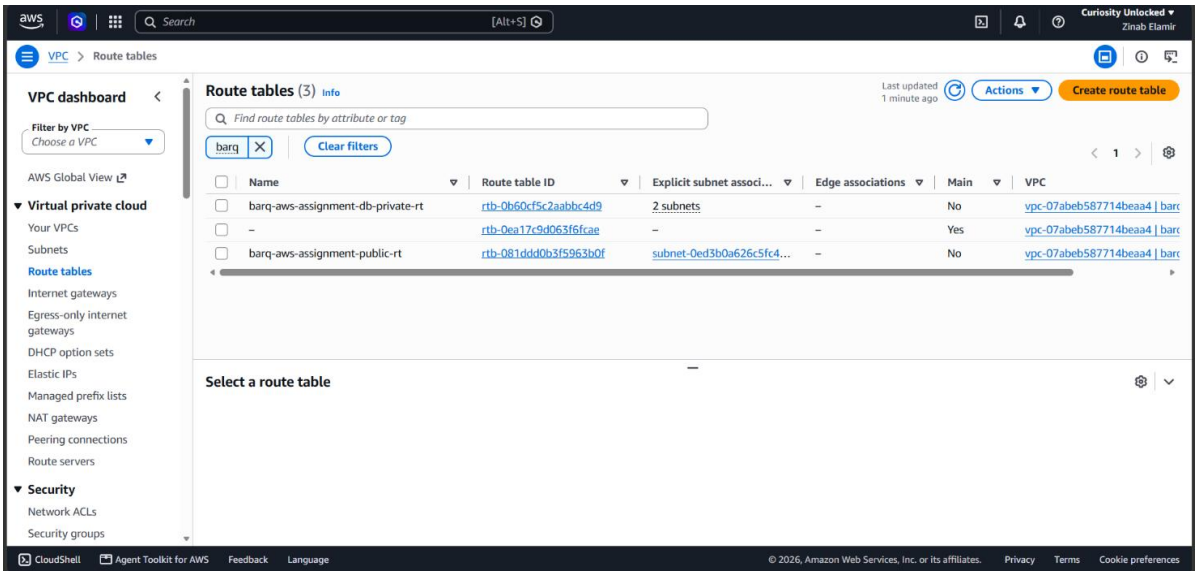


Figure 2: route tables

Figure 3. Three subnets, one public and two private for RDS, all Available.

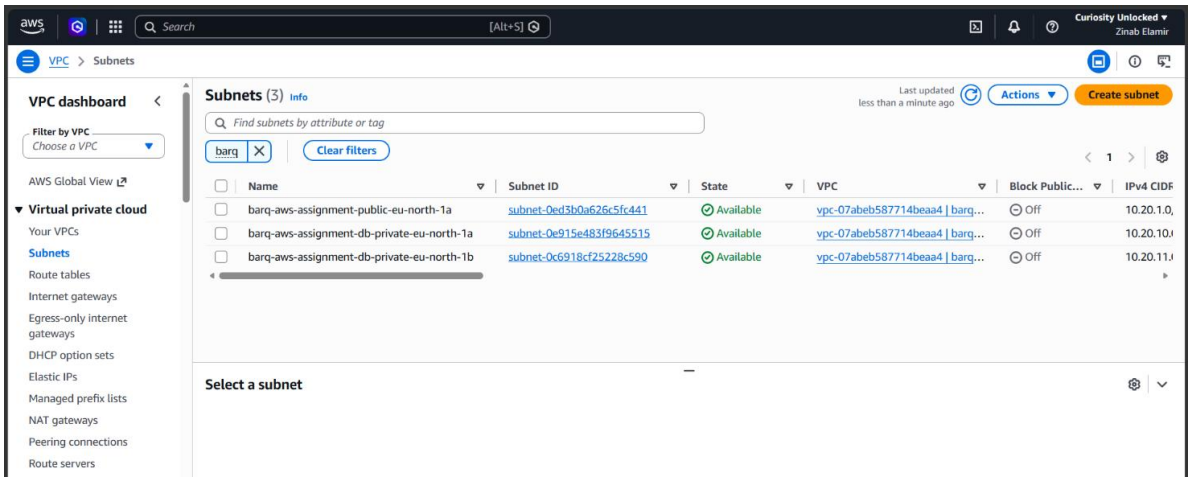


Figure 3: subnets

Figure 4. Internet gateway `barq-aws-assignment-igw` attached to the VPC.

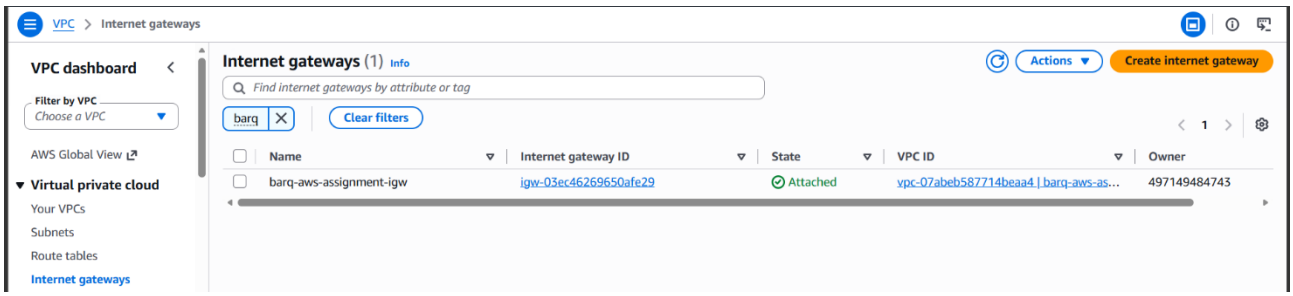


Figure 4: internet gateway

Figure 5. Control plane (c7i-flex.large) and worker (t3.small) running with 3/3 status checks.

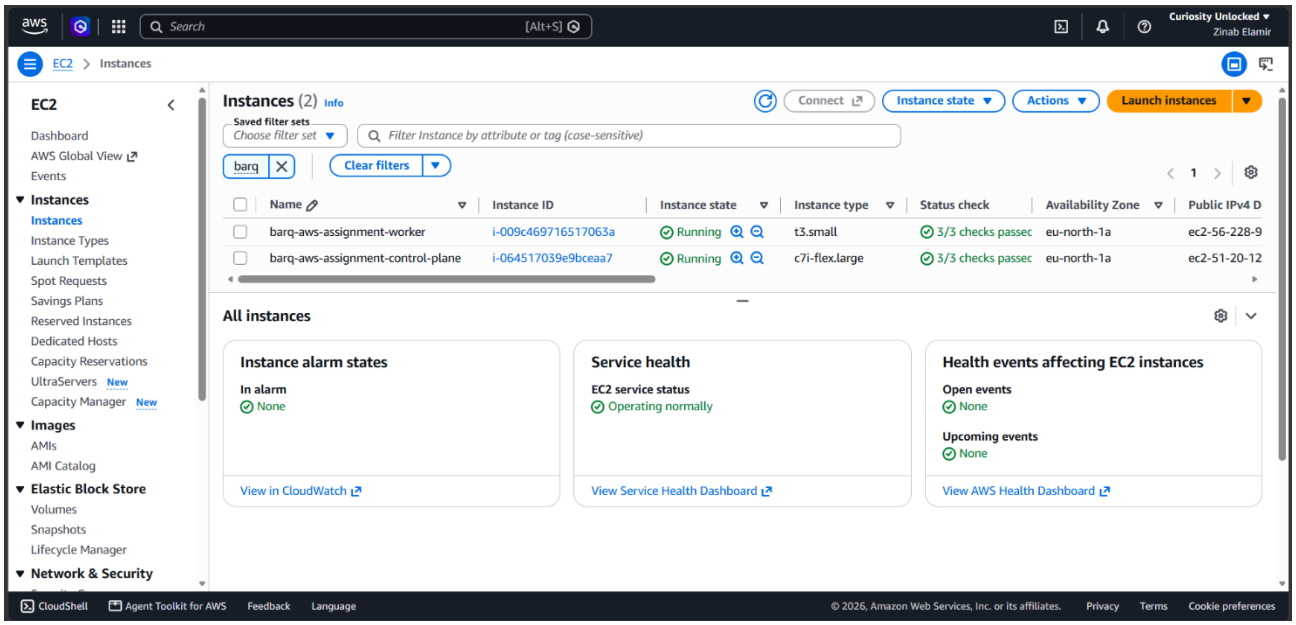


Figure 5: EC2 instances

Figure 6. `terraform init` moving the state to the S3 backend; `terraform state list` shows the managed resources.

```
Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
root@DESKTOP-7QSAAVK:/barq-task/BARQ-AWS-Assignment/terraform# terraform state list
terraform plan
data.aws_ami.ubuntu_2404
data.aws_availability_zones.available
aws_db_instance.postgres
aws_db_subnet_group.main
aws_iam_instance_profile.ec2
aws_iam_role.ec2_ssm
aws_iam_role_policy_attachment.ssm_core
aws_instance.control_plane
aws_instance.worker
aws_internet_gateway.main
aws_route_table.private_db
aws_route_table.public
aws_route_table_association.db_a
aws_route_table_association.db_b
aws_route_table_association.public
```

Figure 6: S3 backend

Figure 7. terraform apply complete (19 added) with the outputs: node IPs, RDS endpoint, database name.

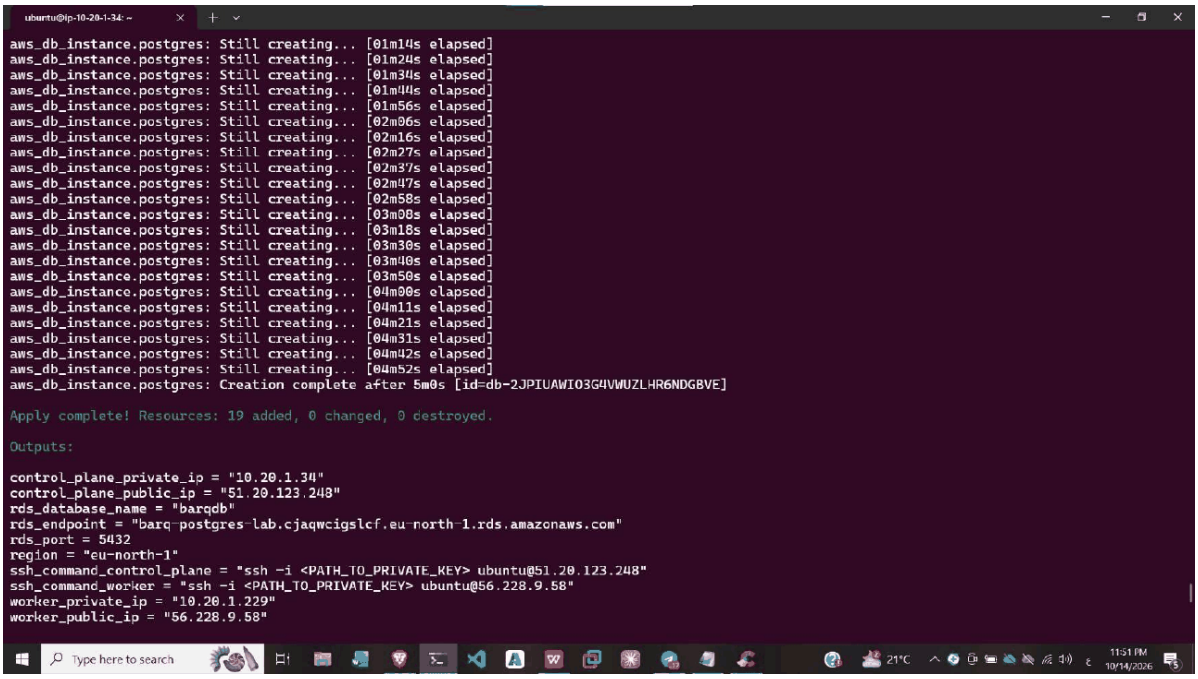


Figure 7: Terraform outputs

Figure 8. Node security group rules: NodePort 30080, all traffic inside the group, API 6443 and SSH 22 from the client range.

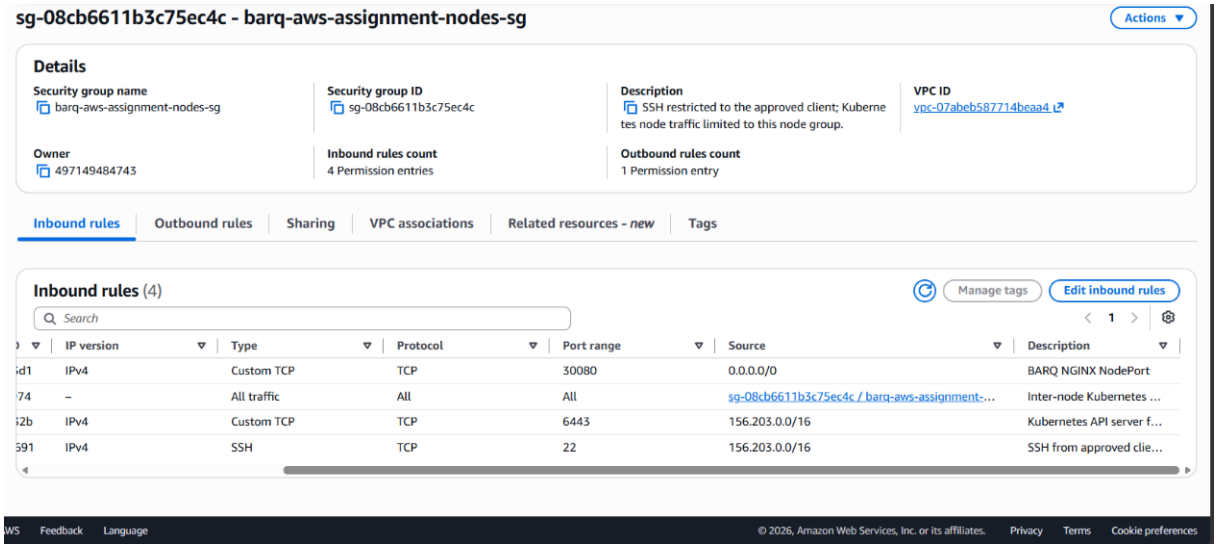


Figure 8: security group

B. Kubernetes cluster built by Ansible

Figure 9. generate-inventory.sh writing hosts.ini from the Terraform outputs.

```
root@DESKTOP-7QSAAVK:/barq-task/BARQ-AWS-Assignment/Ansible# ./generate-inventory.sh ~/.ssh/project.pem
Wrote /barq-task/BARQ-AWS-Assignment/Ansible/hosts.ini
[master]
master-node ansible_host=51.20.123.248 private_ip=10.20.1.34

[workers]
worker-1 ansible_host=56.228.9.58

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=/root/.ssh/project.pem
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=accept-new'
root@DESKTOP-7QSAAVK:/barq-task/BARQ-AWS-Assignment/Ansible#
```

Figure 9: inventory generation

Figure 10. First playbook run: master 25 changed, worker 15 changed, no failures, both nodes Ready.

```
TASK [Store the join command as a fact] ****************************************************************************************************************
ok: [master-node]

PLAY [Worker setup] ************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
ok: [worker-1]

TASK [Join the cluster] ********************************************************************************************************************************
changed: [worker-1]

PLAY [Verify the cluster] ******************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
ok: [master-node]

TASK [Wait until all nodes are Ready] ******************************************************************************************************************
ok: [master-node]

TASK [List nodes] **************************************************************************************************************************************
ok: [master-node]

TASK [Show nodes] **************************************************************************************************************************************
ok: [master-node] => {
    "nodes_out.stdout_lines": [
        "NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUN
TIME",
        "master-node   Ready    control-plane   50s   v1.31.14   10.20.1.34     <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://
2.2.1",
        "worker-1      Ready    <none>          15s   v1.31.14   10.20.1.229    <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://
2.2.1"
    ]
}

PLAY RECAP *********************************************************************************************************************************************
master-node                : ok=38   changed=25   unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
worker-1                   : ok=22   changed=15   unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

Figure 10: Ansible first run

Figure 11. Rerun on the working cluster: `changed=0` on both nodes, Calico tasks skipped.

```
ok: [master-node]

PLAY [Worker setup] ************************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [worker-1]

TASK [Join the cluster] ********************************************************************************************************************************************
ok: [worker-1]

PLAY [Verify the cluster] ******************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [master-node]

TASK [Wait until all nodes are Ready] ******************************************************************************************************************************
ok: [master-node]

TASK [List nodes] **************************************************************************************************************************************************
ok: [master-node]

TASK [Show nodes] **************************************************************************************************************************************************
ok: [master-node] => {
    "nodes_out.stdout_lines": [
        "NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUN
TIME",
        "master-node   Ready    control-plane   31m   v1.31.14   10.20.1.34    <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://
2.2.1",
        "worker-1      Ready    <none>          31m   v1.31.14   10.20.1.229   <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://
2.2.1"
    ]
}

PLAY RECAP *********************************************************************************************************************************************************
master-node                : ok=29   changed=0    unreachable=0    failed=0    skipped=10   rescued=0    ignored=0
worker-1                   : ok=20   changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

Figure 11: Ansible rerun

Figure 12. `kubectl get nodes -o wide`: master-node and worker-1 Ready on v1.31.14.

```
root@DESKTOP-7QSAAVK:/barq-task/BARQ-AWS-Assignment/Ansible# kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
master-node   Ready    control-plane   49m   v1.31.14   10.20.1.34    <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://2.2.1
worker-1      Ready    <none>          48m   v1.31.14   10.20.1.229   <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws   containerd://2.2.1
```

Figure 12: nodes

Figure 13. All `kube-system` pods Running: Calico, CoreDNS, etcd, API server, controller manager, proxy, scheduler.

```
root@DESKTOP-7QSAAVK:/barq-task/BARQ-AWS-Assignment/Ansible# kubectl get pods -n kube-system
NAME                                       READY   STATUS    RESTARTS   AGE
calico-kube-controllers-868cbf9cc-9p7kb    1/1     Running   0          76m
calico-node-bhtjr                          1/1     Running   0          76m
calico-node-bjc54                          1/1     Running   0          76m
coredns-7c65d6cfc9-drnr8                   1/1     Running   0          77m
coredns-7c65d6cfc9-q4pzt                   1/1     Running   0          77m
etcd-master-node                           1/1     Running   0          77m
kube-apiserver-master-node                 1/1     Running   0          77m
kube-controller-manager-master-node        1/1     Running   0          77m
kube-proxy-85bm7                           1/1     Running   0          77m
kube-proxy-ckxtt                           1/1     Running   0          76m
kube-scheduler-master-node                 1/1     Running   0          77m
```

Figure 13: kube-system pods

C. Storage and application deployment

Figure 14. local-path StorageClass installed, `worker-1` labelled `redis-storage=true`, then `deploy.sh` creating the namespace, Secret, ConfigMaps, Services and PVC.

```
$ kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.30/deploy/local-path-storage.yaml
namespace/local-path-storage created
serviceaccount/local-path-provisioner-service-account created
role.rbac.authorization.k8s.io/local-path-provisioner-role created
clusterrole.rbac.authorization.k8s.io/local-path-provisioner-role created
rolebinding.rbac.authorization.k8s.io/local-path-provisioner-bind created
clusterrolebinding.rbac.authorization.k8s.io/local-path-provisioner-bind created
deployment.apps/local-path-provisioner created
storageclass.storage.k8s.io/local-path created
configmap/local-path-config created
$ kubectl get storageclass
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  14s
$ kubectl -n local-path-storage get pods
NAME                                      READY   STATUS    RESTARTS   AGE
local-path-provisioner-dbff48958-mzkwz   1/1     Running   0          23s
$ kubectl label node worker-1 redis-storage=true
node/worker-1 labeled
$ kubectl get nodes --show-labels | grep redis-storage
worker-1      Ready    <none>          21h    v1.31.14   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=work
er-1,kubernetes.io/os=linux,redis-storage=true
$ ./deploy.sh
namespace/barq-project created
secret/barq-secret created
namespace/barq-project unchanged
configmap/barq-config created
configmap/db-init-sql created
configmap/nginx-conf created
service/barq-app created
service/nginx created
service/redis created
persistentvolumeclaim/redis-data created
```

Figure 14: storage and deploy.sh

D. Application verification

Figure 15. First checks through the NodePort: `/` returns the welcome message, `/records` the two seed rows, `/counter` returns 2.

```
$ NODE=51.20.123.248:30080
$ curl -s http://$NODE/ ; echo
{"instance_id":"barq-app-86dccb6656-6f615","message":"Welcome to BARQ Systems - v1","service":"barq-api","version":"2.1.0"}

$ curl -s -i http://$NODE/records
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Thu, 15 Oct 2026 20:10:29 GMT
Content-Type: application/json
Content-Length: 191
Connection: keep-alive
X-Instance-ID: barq-app-86dccb6656-5pqqp
X-Request-ID: 5be5b7faf3c34cf3886839228f16dedc
Cache-Control: no-store

{"instance_id":"barq-app-86dccb6656-5pqqp","records":[{"id":1,"title":"Review service readiness"},{"id":2,"title":"Document the operating procedure"}],"service":"barq-api","version":"
2.1.0"}
$
$ curl -s -i http://$NODE/counter
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Thu, 15 Oct 2026 20:11:01 GMT
Content-Type: application/json
Content-Length: 95
Connection: keep-alive
X-Instance-ID: barq-app-86dccb6656-6f615
X-Request-ID: 1618cfeeb44949f789a3411c37de2e61
Cache-Control: no-store

{"counter":2,"instance_id":"barq-app-86dccb6656-6f615","service":"barq-api","version":"2.1.0"}
```

Figure 15: first application checks

Figure 16. All six endpoints answer HTTP 200 through NGINX; `/ready` reports PostgreSQL and Redis ready.

```
$ for p in / /health /ready /instance /records /counter; do
  echo "== $p"; curl -s -w "\nHTTP %{http_code}\n" http://$NODE$p
done> >
== /
{"instance_id":"barq-app-86dccb6656-6f6l5","message":"Welcome to BARQ Systems - v1","service":"barq-api","version":"2.1.0"}

HTTP 200
== /health
{"instance_id":"barq-app-86dccb6656-6f6l5","service":"barq-api","status":"alive","version":"2.1.0"}

HTTP 200
== /ready
{"dependencies":{"postgres":"ready","redis":"ready"},"instance_id":"barq-app-86dccb6656-5pqqp","service":"barq-api","status":"ready","version":"2.1.0"}

HTTP 200
== /instance
{"instance_id":"barq-app-86dccb6656-5pqqp","service":"barq-api","status":"ok","version":"2.1.0"}

HTTP 200
== /records
{"instance_id":"barq-app-86dccb6656-5pqqp","records":[{"id":1,"title":"Review service readiness"},{"id":2,"title":"Document the operating procedure"}],"service":"barq-api","version":"
2.1.0"}

HTTP 200
== /counter
{"counter":3,"instance_id":"barq-app-86dccb6656-5pqqp","service":"barq-api","version":"2.1.0"}

HTTP 200
$
```

Figure 16: six endpoints

Figure 17. `POST /records` creates `proof-rds-1` (id 3) and `GET /records` returns it.

```
$ curl -s -X POST http://$NODE/records -H 'Content-Type: application/json' -d '{"title":"proof-rds-1"}'
curl -s http://$NODE/records{"instance_id":"barq-app-86dccb6656-5pqqp","record":{"id":3,"title":"proof-rds-1"},"service":"barq-api","version":"2.1.0"}
$
{"instance_id":"barq-app-86dccb6656-5pqqp","records":[{"id":1,"title":"Review service readiness"},{"id":2,"title":"Document the operating procedure"},{"id":3,"title":
"proof-rds-1"}],"service":"barq-api","version":"2.1.0"}
$
$
```

Figure 17: create a record

Figure 18. `psql` from a temporary pod straight to RDS with `sslmode=require`: the same three rows.

```
$ kubectl -n barq-project run psql-check --rm -it --restart=Never --image=postgres:16-alpine \
  --env="PGPASSWORD=$(kubectl -n barq-project get secret barq-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d)" \
  -- psql "host=$(kubectl -n barq-project get cm barq-config -o jsonpath='{.data.DB_HOST}') dbname=barqdb user=$(kubectl -n barq-project get secret barq-secret -o jso
npath='{.data.DB_USER}' | base64 -d) sslmode=require" \
  -c "SELECT * FROM records;"> > >
 id |             title
----+-------------------------------
  1 | Review service readiness
  2 | Document the operating procedure
  3 | proof-rds-1
(3 rows)

pod "psql-check" deleted
$
```

Figure 18: read the record from RDS

Figure 19. Both Flask pods deleted; after the rollout `/records` still returns all three rows.

```
$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-5pqqp   1/1     Running   0          68m
barq-app-86dccb6656-6f6l5   1/1     Running   0          68m
$ kubectl -n barq-project delete pod -l app=barq-app
pod "barq-app-86dccb6656-5pqqp" deleted
pod "barq-app-86dccb6656-6f6l5" deleted
^C$
$
$ kubectl -n barq-project rollout status deployment/barq-app
deployment "barq-app" successfully rolled out
$ curl -s http://$NODE/records
{"instance_id":"barq-app-86dccb6656-kqxzc","records":[{"id":1,"title":"Review service readiness"},{"id":2,"title":"Document the operating procedure"},{"id":3,"title":
"proof-rds-1"}],"service":"barq-api","version":"2.1.0"}
$
```

Figure 19: Flask pods deleted

Figure 20. The replacement Flask pods, created by the Deployment, running.

```
$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-kqxzc   1/1     Running   0          109s
barq-app-86dccb6656-x8lpf   1/1     Running   0          109s
$
```

Figure 20: replacement Flask pods

Figure 21. Counter at 4 and 5, Redis pod deleted, counter continues at 6.

```
$ curl -s http://$NODE/counter ; echo
{"counter":4,"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","version":"2.1.0"}

$ curl -s http://$NODE/counter ; echo
{"counter":5,"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","version":"2.1.0"}

$ sleep 3
$ kubectl -n barq-project get pvc redis-data
NAME         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
redis-data   Bound    pvc-de3c83c1-870a-4ab0-8029-b6084bb696c9   1Gi        RWO            local-path     <unset>                 84m
$
$ kubectl -n barq-project delete pod -l app=redis
pod "redis-87b458785-dd4fz" deleted
$ kubectl -n barq-project rollout status deployment/redis
deployment "redis" successfully rolled out
$ curl -s http://$NODE/counter ; echo
{"counter":6,"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","version":"2.1.0"}

$
```

Figure 21: counter before and after Redis restart

Figure 22. PVC `redis-data` still Bound; the new Redis pod is running.

```
$ kubectl -n barq-project get pvc redis-data
NAME         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
redis-data   Bound    pvc-de3c83c1-870a-4ab0-8029-b6084bb696c9   1Gi        RWO            local-path     <unset>                 84m
$ kubectl -n barq-project get pods -l app=redis
NAME                     READY   STATUS    RESTARTS   AGE
redis-87b458785-2tdcx   1/1     Running   0          86s
$
```

Figure 22: PVC after the first Redis deletion

Figure 23. Second test: counter 7 and 8, Redis pod deleted again, counter continues at 9.

```
$
$ kubectl -n barq-project get pods -l app=redis
NAME                     READY   STATUS    RESTARTS   AGE
redis-87b458785-2tdcx    1/1     Running   0          114s
$ curl -s http://$NODE/counter ; echo
{"counter":7,"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","version":"2.1.0"}

$ curl -s http://$NODE/counter ; echo
{"counter":8,"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","version":"2.1.0"}

$ kubectl -n barq-project delete pod -l app=redis
pod "redis-87b458785-2tdcx" deleted
$ kubectl -n barq-project rollout status deployment/redis
deployment "redis" successfully rolled out
$ curl -s http://$NODE/counter ; echo
{"counter":9,"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","version":"2.1.0"}

$ kubectl -n barq-project get pods -l app=redis
NAME                     READY   STATUS    RESTARTS   AGE
redis-87b458785-5rp9m    1/1     Running   0          21s
$
```

Figure 23: second Redis restart

Figure 24. PVC `redis-data` (1 Gi, ReadWriteOnce, local-path) still Bound.

```
$ kubectl -n barq-project get pvc redis-data
NAME         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
redis-data   Bound    pvc-de3c83c1-870a-4ab0-8029-b6084bb696c9   1Gi        RWO            local-path     <unset>                 88m
$
```

Figure 24: PVC after the second deletion

Figure 25. Twelve repeated requests through NGINX alternate between the two Flask replicas.

```
$
$ for i in $(seq 1 12); do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-x8lpf","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-kqxzc","service":"barq-api","status":"ok","version":"2.1.0"}
```

Figure 25: both replicas answer

Figure 26. One Flask pod deleted; the Deployment creates a replacement and 2/2 pods run again.

```
$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-ch5q7   1/1     Running   0          73s
barq-app-86dccb6656-x8lpf   1/1     Running   0          17m
$ kubectl -n barq-project delete pod barq-app-86dccb6656-x8lpf
pod "barq-app-86dccb6656-x8lpf" deleted
^C$
$
$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-9mj6d   1/1     Running   0          54s
barq-app-86dccb6656-ch5q7   1/1     Running   0          3m27s
$
```

Figure 26: pod replacement

Figure 27. A 200-request loop after the replacement: answers keep alternating between the two pods.

```
$
$ for i in $(seq 1 200); do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-9mj6d","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}
```

Figure 27: load across both replicas

Figure 28. Service DNS from a pod pinned to `master-node`: CoreDNS resolves `barq-app` and `redis`, and the call to Flask succeeds.

```
$ for N in master-node worker-1; do
  kubectl -n barq-project run dns-$N --rm -it --restart=Never --image=busybox:1.36 \
    --overrides="{\"spec\":{\"nodeName\":\"$N\",\"tolerations\":[{\"operator\":\"Exists\"}]}}" \
    -- sh -c "echo '--- on $N'; nslookup barq-app; nslookup redis; wget -qO- http://barq-app:8080/instance"
done> > > >
--- on master-node
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   barq-app.barq-project.svc.cluster.local
Address: 10.102.233.31

** server can't find barq-app.cluster.local: NXDOMAIN

** server can't find barq-app.cluster.local: NXDOMAIN


** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.eu-north-1.compute.internal: NXDOMAIN

** server can't find barq-app.eu-north-1.compute.internal: NXDOMAIN

Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   redis.barq-project.svc.cluster.local
Address: 10.106.38.234

** server can't find redis.svc.cluster.local: NXDOMAIN
```

Figure 28: DNS from the control plane

Figure 29. The same test from a pod pinned to `worker-1`.

```
** server can't find redis.cluster.local: NXDOMAIN

** server can't find redis.eu-north-1.compute.internal: NXDOMAIN

** server can't find redis.eu-north-1.compute.internal: NXDOMAIN

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}
pod "dns-master-node" deleted
--- on worker-1
Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find barq-app.eu-north-1.compute.internal: NXDOMAIN


Name:   barq-app.barq-project.svc.cluster.local
Address: 10.102.233.31

** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.cluster.local: NXDOMAIN

** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.cluster.local: NXDOMAIN

** server can't find barq-app.eu-north-1.compute.internal: NXDOMAIN

Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find redis.svc.cluster.local: NXDOMAIN

Name:   redis.barq-project.svc.cluster.local
```

Figure 29: DNS from the worker

Figure 30. End of the worker test: Flask JSON returned, temporary pod removed.

```
Name:   barq-app.barq-project.svc.cluster.local
Address: 10.102.233.31

** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.cluster.local: NXDOMAIN

** server can't find barq-app.svc.cluster.local: NXDOMAIN

** server can't find barq-app.cluster.local: NXDOMAIN

** server can't find barq-app.eu-north-1.compute.internal: NXDOMAIN

Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find redis.svc.cluster.local: NXDOMAIN

Name:   redis.barq-project.svc.cluster.local
Address: 10.106.38.234

** server can't find redis.eu-north-1.compute.internal: NXDOMAIN


** server can't find redis.cluster.local: NXDOMAIN

** server can't find redis.svc.cluster.local: NXDOMAIN

** server can't find redis.eu-north-1.compute.internal: NXDOMAIN

** server can't find redis.cluster.local: NXDOMAIN

{"instance_id":"barq-app-86dccb6656-ch5q7","service":"barq-api","status":"ok","version":"2.1.0"}
pod "dns-worker-1" deleted
$
```

Figure 30: DNS test finished

E. Image update and rollback

Figure 31. The application version in `server.py` edited so the new image is visibly different.

```
  GNU nano 7.2                                                    server.py *
"""Small Flask/PostgreSQL/Redis service for the BARQ assessment."""
import json
import os
import re
import time
import uuid
from datetime import datetime, timezone
import psycopg
import redis
from flask import Flask, g, jsonify, request
from werkzeug.exceptions import HTTPException

VERSION = "2.1.0"

def log_event(level, event, **fields):
    print(json.dumps({"timestamp": datetime.now(timezone.utc).isoformat(timespec="milliseconds"),
                      "level": level, "service": "barq-api", "event": event, **fields}), flush=True)

class Dependencies:
    def __init__(self, database_url, redis_url):
        self.database_url = database_url
        self.cache = redis.Redis.from_url(redis_url, socket_connect_timeout=2,
                                          socket_timeout=2, decode_responses=True)

    def query(self, statement, params=()):
        with psycopg.connect(self.database_url, connect_timeout=2,
                             options="-c statement_timeout=2000") as connection:
            with connection.cursor() as cursor:
                cursor.execute(statement, params)
                return cursor.fetchall()

    def database_ready(self):

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

Figure 31: version change

Figure 32. Build tooling installed on the control plane to build the test image.

```
$
$ sudo apt-get update && sudo apt-get install -y podman
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1222 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [246 kB]
Hit:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.31/deb  InRelease
Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1704 kB]
Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [342 kB]
Fetched 3765 kB in 1s (4770 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  aardvark-dns buildah catatonit common containernetworking-plugins fuse-overlayfs golang-github-containers-common golang-github-containers-image libslirp0 libsubid4 netavark passt
  slirp4netns uidmap
Suggested packages:
  containers-storage docker-compose
The following NEW packages will be installed:
  aardvark-dns buildah catatonit common containernetworking-plugins fuse-overlayfs golang-github-containers-common golang-github-containers-image libslirp0 libsubid4 netavark passt
  podman slirp4netns uidmap
0 upgraded, 15 newly installed, 0 to remove and 7 not upgraded.
Need to get 32.1 MB of archives.
After this operation, 130 MB of additional disk space will be used.
Get:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 netavark amd64 1.4.0-4 [1666 kB]
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 aardvark-dns amd64 1.4.0-5 [881 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libsubid4 amd64 1:4.13+dfsg1-4ubuntu3.2 [23.4 kB]
Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 uidmap amd64 1:4.13+dfsg1-4ubuntu3.2 [26.0 kB]
Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 containernetworking-plugins amd64 1.1.1+ds1-3ubuntu0.24.04.3 [7203 kB]
Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 golang-github-containers-image all 5.29.2-2 [33.4 kB]
Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 golang-github-containers-common all 0.57.4+ds1-2ubuntu0.2 [39.9 kB]
Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 buildah amd64 1.33.7+ds1-1ubuntu0.24.04.3 [8148 kB]
```

Figure 32: build tooling

Figure 33. Baseline on image `6ca3443` (version 2.1.0), then the `v3` build starts.

```
$ NODE=51.20.123.248:30080
$ kubectl -n barq-project get deploy barq-app -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
docker.io/zinabelamir/barq-app:6ca3443
$ for i in 1 2 3 4; do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-86dccb6656-2b5dl","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-2b5dl","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-fwmmt","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-2b5dl","service":"barq-api","status":"ok","version":"2.1.0"}

$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-2b5dl   1/1     Running   0          11m
barq-app-86dccb6656-fwmmt   1/1     Running   0          11m
$ kubectl -n barq-project rollout history deployment/barq-app
deployment.apps/barq-app
REVISION  CHANGE-CAUSE
1         <none>
3         update to v2
4         <none>

$ sudo podman build -t docker.io/zinabelamir/barq-app:v3 .
STEP 1/10: FROM python:3.12-slim-bookworm@sha256:782412e85d0f0984994c290652577d4018aff08145c85b262bb63dc0c7522254
STEP 2/10: ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
--> Using cache 74dd024f576a5da3d086b8802418dacfec6352e2877f16cfdc9467720da6139c
--> 74dd024f576a
STEP 3/10: WORKDIR /srv
--> Using cache 14062942fad47be198f68b0b4f944d5c0d2fae83950e1d0d21ddf58d20ea573a
--> 14062942fad4
STEP 4/10: RUN groupadd --gid 10001 app && useradd --uid 10001 --gid app --no-create-home app
--> Using cache c844e62ff9ffd753b516a860bcdee4d08ecc963c49a6a80c74255c3072ce53f0
--> c844e62ff9ff
STEP 5/10: COPY requirements.txt ./requirements.txt
--> Using cache 1b503af9e2b0182d63e979c9eaa372388cc0489f421573df928a4d9d646a25c6
```

Figure 33: baseline and build

Figure 34. v3 pushed and rolled out; every response reports version 2.2.0.

```
$ sudo podman push docker.io/zinabelamir/barq-app:v3
Getting image source signatures
Copying blob 0213c4c9838a skipped: already exists
Copying blob ec613f6df892 skipped: already exists
Copying blob 35ef2f664a5c skipped: already exists
Copying blob a8ac7f6c67ab skipped: already exists
Copying blob d4169fd0e517 skipped: already exists
Copying blob 268fd66f673d skipped: already exists
Copying blob f95f9f053d10 done   |
Copying blob 6ab1cec98c30 skipped: already exists
Copying config 4d0f02acc5 done   |
Writing manifest to image destination
$ kubectl -n barq-project set image deployment/barq-app app=docker.io/zinabelamir/barq-app:v3
deployment.apps/barq-app image updated
$ kubectl -n barq-project annotate deployment/barq-app kubernetes.io/change-cause="update to v3 (version 2.2.0)" --overwrite
deployment.apps/barq-app annotated
$ kubectl -n barq-project rollout status deployment/barq-app
deployment "barq-app" successfully rolled out
$ kubectl -n barq-project get deploy barq-app -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
docker.io/zinabelamir/barq-app:v3
$ for i in 1 2 3 4 5 6; do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-7b8cb4f5bc-bkzgt","service":"barq-api","status":"ok","version":"2.2.0"}

{"instance_id":"barq-app-7b8cb4f5bc-hwh77","service":"barq-api","status":"ok","version":"2.2.0"}

{"instance_id":"barq-app-7b8cb4f5bc-bkzgt","service":"barq-api","status":"ok","version":"2.2.0"}

{"instance_id":"barq-app-7b8cb4f5bc-bkzgt","service":"barq-api","status":"ok","version":"2.2.0"}

{"instance_id":"barq-app-7b8cb4f5bc-hwh77","service":"barq-api","status":"ok","version":"2.2.0"}

{"instance_id":"barq-app-7b8cb4f5bc-hwh77","service":"barq-api","status":"ok","version":"2.2.0"}

$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-7b8cb4f5bc-bkzgt   1/1     Running   0          45s
```

Figure 34: update to v3

Figure 35. Rollout history shows the update; kubectl rollout undo starts the rollback.

```
$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-7b8cb4f5bc-bkzgt   1/1     Running   0          45s
barq-app-7b8cb4f5bc-hwh77   1/1     Running   0          55s
$ kubectl -n barq-project rollout history deployment/barq-app
deployment.apps/barq-app
REVISION  CHANGE-CAUSE
1         <none>
3         update to v2
4         <none>
5         update to v3 (version 2.2.0)

$ kubectl -n barq-project rollout undo deployment/barq-app
deployment.apps/barq-app rolled back
$ kubectl -n barq-project rollout status deployment/barq-app
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
deployment "barq-app" successfully rolled out
$ kubectl -n barq-project annotate deployment/barq-app kubernetes.io/change-cause="rollback to 6ca3443" --overwrite
deployment.apps/barq-app annotated
$ kubectl -n barq-project get deploy barq-app -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
docker.io/zinabelamir/barq-app:6ca3443
$ for i in 1 2 3 4 5 6; do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"ok","version":"2.1.0"}
```

Figure 35: rollback

Figure 36. Rollback verified: image is `6ca3443` again, responses report 2.1.0, history records the rollback.

```
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
deployment "barq-app" successfully rolled out
$ kubectl -n barq-project annotate deployment/barq-app kubernetes.io/change-cause="rollback to 6ca3443" --overwrite
deployment.apps/barq-app annotated
$ kubectl -n barq-project get deploy barq-app -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
docker.io/zinabelamir/barq-app:6ca3443
$ for i in 1 2 3 4 5 6; do curl -s http://$NODE/instance; echo; done
{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-cbg85","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"ok","version":"2.1.0"}

{"instance_id":"barq-app-86dccb6656-qjcr9","service":"barq-api","status":"ok","version":"2.1.0"}

$ kubectl -n barq-project rollout history deployment/barq-app
deployment.apps/barq-app
REVISION  CHANGE-CAUSE
1         <none>
3         update to v2
5         update to v3 (version 2.2.0)
6         rollback to 6ca3443

$ kubectl -n barq-project get pods -l app=barq-app
NAME                        READY   STATUS    RESTARTS   AGE
barq-app-86dccb6656-cbg85   1/1     Running   0          114s
barq-app-86dccb6656-qjcr9   1/1     Running   0          104s
$
```

Figure 36: rollback verified

Figure 37. Hardened Flask Deployment applied: kubectl apply -k changed only barq-app, the rolling update finished with no downtime, and /ready reports PostgreSQL and Redis ready.

```
$ kubectl apply -k manifests/k8s
namespace/barq-project unchanged
configmap/barq-config unchanged
configmap/db-init-sql unchanged
configmap/nginx-conf unchanged
service/barq-app unchanged
service/nginx unchanged
service/redis unchanged
persistentvolumeclaim/redis-data unchanged
deployment.apps/barq-app configured
deployment.apps/nginx unchanged
deployment.apps/redis unchanged
job.batch/db-init unchanged
networkpolicy.networking.k8s.io/app-ingress unchanged
networkpolicy.networking.k8s.io/nginx-ingress unchanged
networkpolicy.networking.k8s.io/redis-ingress unchanged

$ kubectl -n barq-project rollout status deploy/barq-app --timeout=120s
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "barq-app" rollout to finish: 1 old replicas are pending termination...
deployment "barq-app" successfully rolled out

$ curl http://51.20.123.248:30080/ready
{"dependencies":{"postgres":"ready","redis":"ready"},"instance_id":"barq-app-7854b575df-vk9gw","service":"barq-api","s
  tatus":"ready","version":"2.1.0"}
```

Figure 37: security context rollout (terminal output, 16 October 2026)

Figure 38. Writes still work after hardening: POST /records created id 4 and the Redis counter advanced to 11.

```
$ curl -s -X POST http://51.20.123.248:30080/records -H 'Content-Type: application/json' -d
  '{"title":"proof-hardened"}'
{"instance_id":"barq-app-7854b575df-rl4m5","record":{"id":4,"title":"proof-hardened"},"service":"barq-api","version":"
  2.1.0"}

$ curl -s http://51.20.123.248:30080/counter
{"counter":11,"instance_id":"barq-app-7854b575df-vk9gw","service":"barq-api","version":"2.1.0"}
```

Figure 38: write checks after hardening (terminal output)

Figure 39. The hardening is in effect: one Flask pod per node is Running, the container has the securityContext, the root file system is read-only and the process runs as uid 10001.

```
$ kubectl -n barq-project get pods -l app=barq-app -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP              NODE          NOMINATED NODE   READINESS GATES
barq-app-7854b575df-rl4m5   1/1     Running   0          64s   10.244.77.146   master-node   <none>           <none>
barq-app-7854b575df-vk9gw   1/1     Running   0          69s   10.244.226.88   worker-1      <none>           <none>

$ kubectl -n barq-project get deploy barq-app -o jsonpath='{.spec.template.spec.containers[0].securityContext}{"\n"}'
{"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"]},"readOnlyRootFilesystem":true,"runAsNonRoot":true,"runAsUser":
  10001}

$ kubectl -n barq-project exec deploy/barq-app -- sh -c 'touch /test 2>&1; id'
touch: cannot touch '/test': Read-only file system
uid=10001(app) gid=10001(app) groups=10001(app)
```

Figure 39: securityContext proof (terminal output)

CI/CD with GitHub Actions and GitOps with Argo CD

A change to the application now travels from Git to the cluster without a manual step. GitHub Actions checks and builds the image, pushes it to Docker Hub with a fixed tag, and writes that tag into the manifests; Argo CD, running on the control-plane node, sees the commit and rolls the Deployment. Nothing secret is stored in Git: Docker Hub credentials are GitHub repository secrets and the database credentials stay in the Kubernetes Secret `barq-secret`.

One workflow, seven jobs

The pipeline is one file, `.github/workflows/cicd.yml`, so the unit tests and the image build run once per push instead of once per workflow. It starts on a push or pull request to `aws-terraform-ansible` that touches app/, database/, nginx/, tests/, Dockerfile, requirements, docker-compose.yml, validate.py, manifests/, terraform/ or the workflows; a README, documentation or screenshot change starts nothing.

Job	What it does
`changes`	Decides whether the application image is affected (app/, database/, nginx/, Dockerfile, .dockerignore, requirements, docker-compose.yml, .env.example, validate.py).
`unit-tests`	Python unit tests.
`build-start-validate`	Builds the compose stack, starts it and runs `validate.py` against it.
`static-checks`	Renders the manifests with kustomize and validates them with kubeconform; `bash -n` on deploy.sh; Trivy secret scan (fails on findings); Trivy configuration scan (report only).
`terraform-checks`	`terraform fmt -check`, `init -backend=false` and `validate`.
`image`	Runs only on a push to the branch, only when `changes` says the image is affected, and only after the four checks above pass. Builds the image once, scans that same image with Trivy (a fixable CRITICAL stops the run) and pushes `zinabelamir/barq-app:<short-sha>` to Docker Hub.
`deploy-tag`	Writes the new tag into `manifests/k8s/kustomization.yaml` and commits it as github-actions[bot].

A change to Terraform, to the manifests or to the tests runs the checks but builds and pushes no image, and a pull request never pushes anything, because only the application files end up inside the image. Figures 40 to 43 were taken from the first version, which used two workflow files (ci.yml and docker-push.yml); they were merged into cicd.yml afterwards (Figure 46).

What happens on a push to app/

1. The push starts the pipeline; the unit tests, the compose test, the manifest and secret checks and the Terraform checks run in parallel.
2. When they all pass and the image is affected, the `image` job builds it, scans it and pushes the short commit SHA as the only tag (no `latest`, no moving tag, because the Deployment uses `imagePullPolicy: IfNotPresent`).
3. The `deploy-tag` job replaces `newTag` in the kustomization and commits it. That commit only touches manifests/, so it does not start another image build.
4. Argo CD (application `barq`, path `manifests/k8s`, branch `aws-terraform-ansible`, automated sync with prune and self-heal) detects the new commit and applies it.
5. Kubernetes performs a rolling update (`maxUnavailable: 0`, `maxSurge: 1`), so the application keeps answering while the pods are replaced.

Design decisions

- Argo CD is pinned to the control-plane node (nodeSelector plus the control-plane toleration). worker-1 has 2 GiB and already runs NGINX, Flask and Redis; master-node has 4 GiB with about 2.5 GiB free.
- Self-heal is on, so a manual `kubectl set image` is reverted by Argo CD; rollout and rollback tests are done by changing the tag in Git.
- The Secret `barq-secret` and the Retain patch on the Redis PV are not in Git, so Argo CD neither creates nor prunes them.
- The application no longer logs `DATABASE_URL`: at start-up it logs only whether the database and Redis are configured, which is listed under closed gaps in the Security section.

Problems met and fixes

Symptom	Cause	Fix
`terraform fmt -check` failed on main.tf (Figure 40)	main.tf was not formatted after a manual edit	`terraform fmt` in terraform/, then commit
Image scan failed with 3 CRITICAL findings in perl-base (CVE-2026-13221, CVE-2026-42496, CVE-2026-8376), fixed in 5.36.0-7+deb12u4 (Figure 40)	The digest-pinned base image still carried perl-base 5.36.0-7+deb12u3	Dockerfile updated so the fixed package is installed; CI and Docker Push then passed (Figures 41 and 43)
`./install-argocd.sh`: Permission denied	The script was not executable	`chmod +x install-argocd.sh`
Argo CD pods could be scheduled on the 2 GiB worker	No toleration for the control-plane taint, so the scheduler had only worker-1	The install script patches every Argo CD workload to run on master-node and prints `get pods -o wide` to prove it

Figure 40. First CI run on the pipeline files: the gate did its job. terraform-checks failed on formatting and image-scan failed on three fixable CRITICAL vulnerabilities in the base image; unit tests, the compose stack and the manifest checks passed.

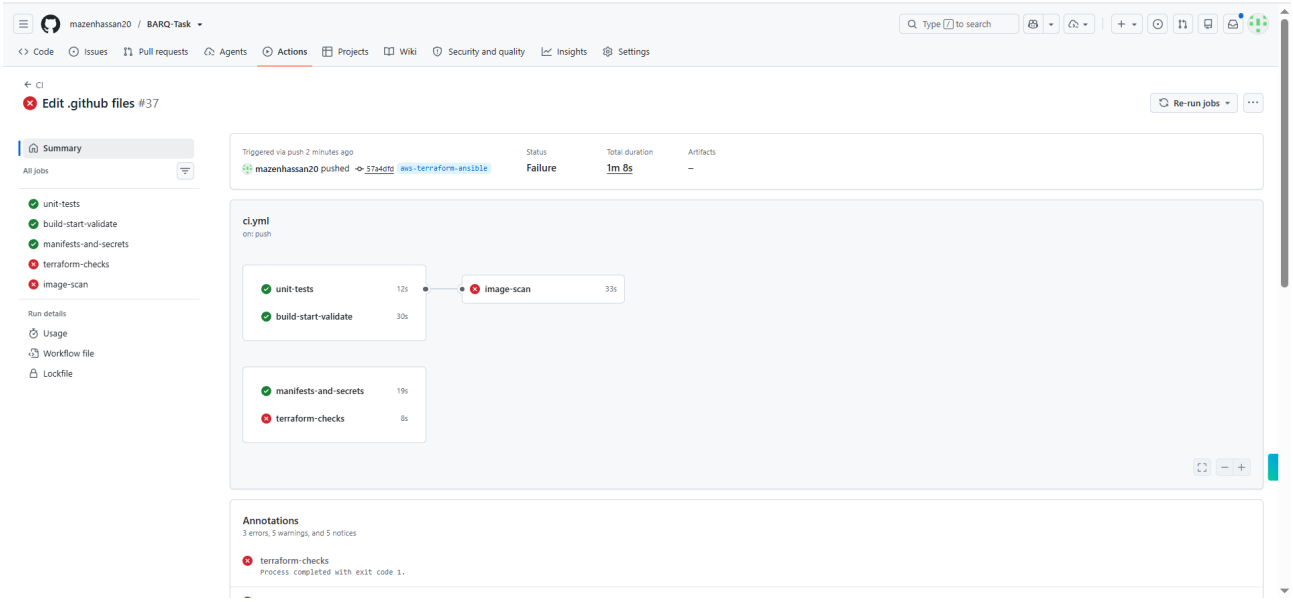


Figure 40: CI run #37, failed gates

Figure 41. After the fixes, all five CI jobs pass: unit-tests, build-start-validate, manifests-and-secrets, terraform-checks and image-scan.

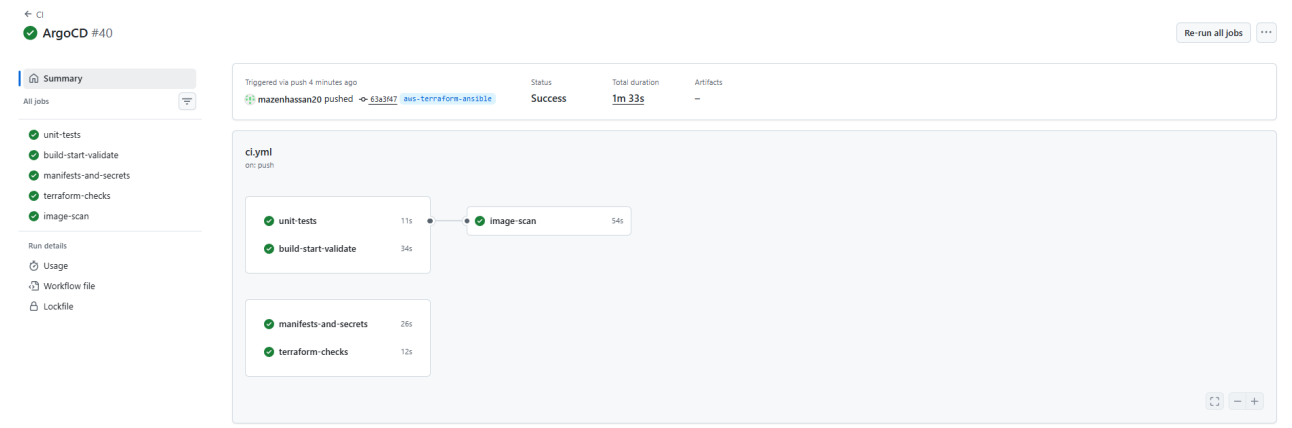


Figure 41: CI run #40, all jobs green

Figure 42. Workflow run history on the branch: Docker Push does not run for the workflow-only commit, and runs together with CI when app files change.

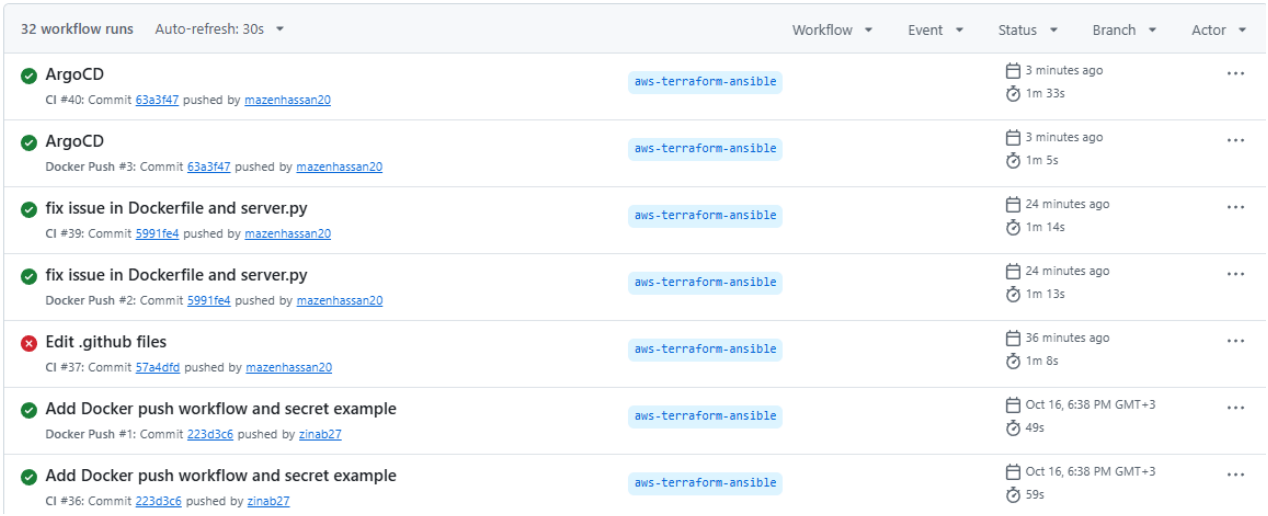


Figure 42: workflow runs on aws-terraform-ansible

Figure 43. Docker Push run: test, build-push and deploy-tag all succeed; the image is pushed with the short-SHA tag 63a3f47 and a digest.

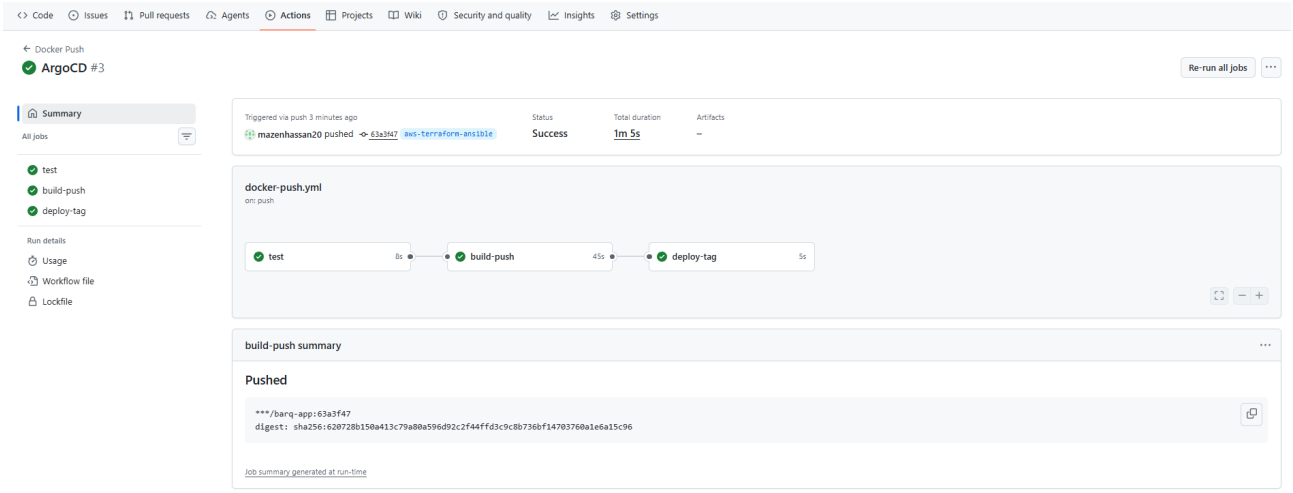


Figure 43: Docker Push run #3

Figure 44. Argo CD application barq: Healthy and Synced, repository BARQ-Task, branch aws-terraform-ansible, path manifests/k8s, namespace barq-project.

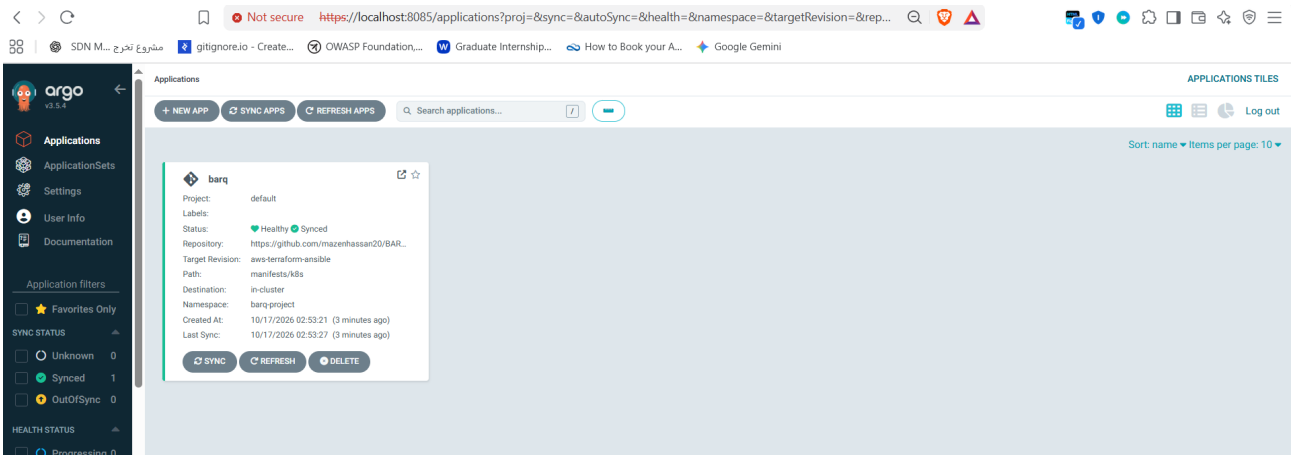


Figure 44: Argo CD application list

Figure 45. Argo CD resource tree of the application: Synced to commit 5991fe4, last sync succeeded; the Deployments, ReplicaSets, pods, Services, PVC, Job and NetworkPolicies are all healthy.

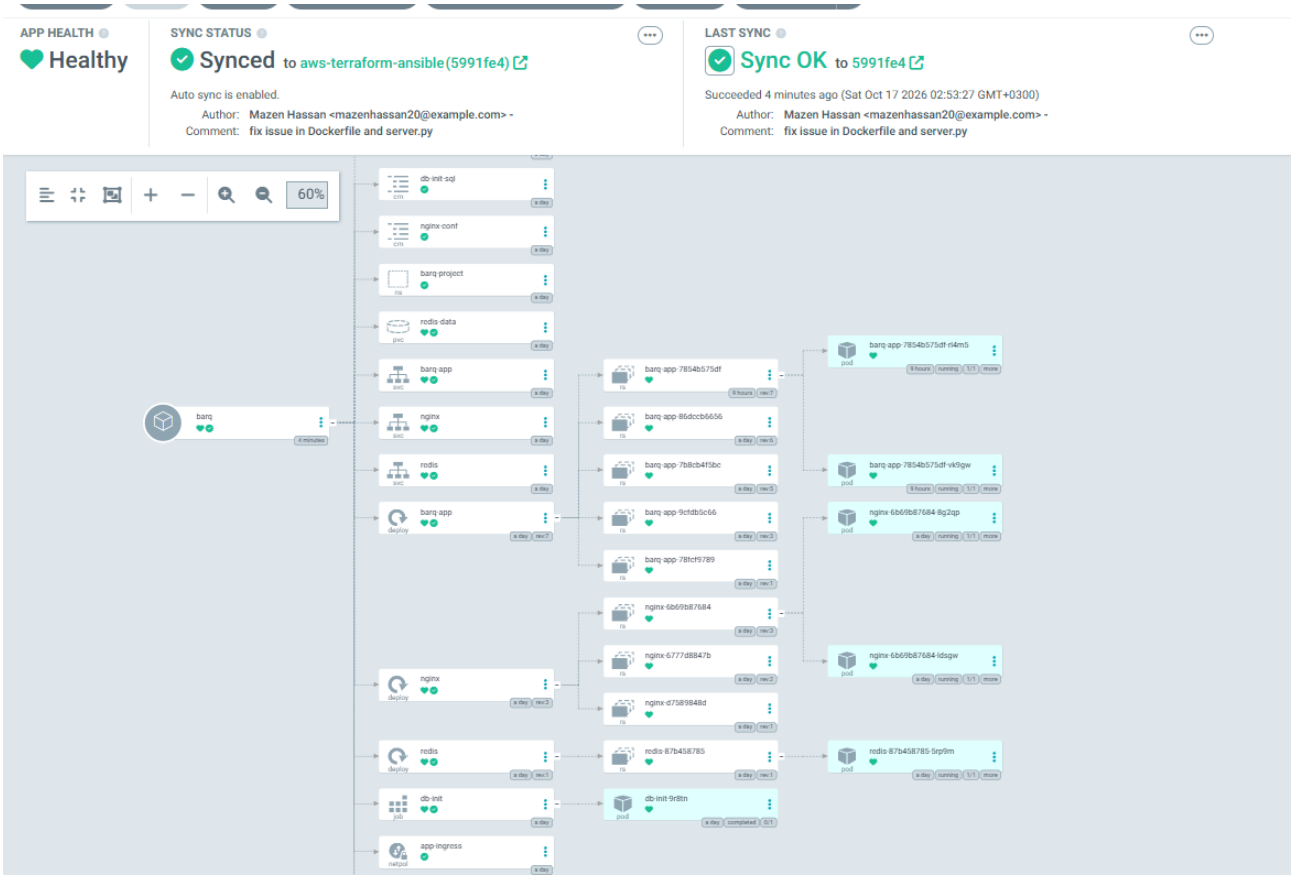


Figure 45: Argo CD application tree

Figure 46. The merged workflow cicd.yml, first run: changes, unit-tests, build-start-validate, static-checks and terraform-checks pass, then image and deploy-tag; the published image is tagged with the commit short SHA.

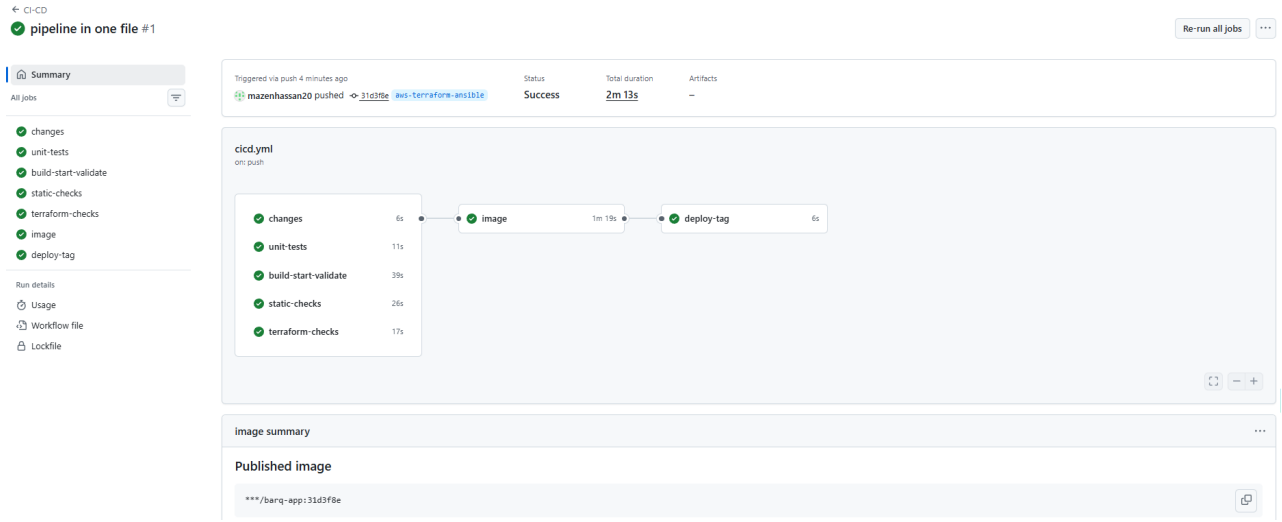


Figure 46: cicd.yml run #1, all seven jobs green